

神威高性能计算机

调试工具用户手册

二〇二〇年十二月

序

神威高性能计算机软件调试工具包括 swpdb 远程调试工具、轻量级调试工具集以及 penv_debug 并行调试工具 GUI。

swpdb 远程调试工具支持调试正在运行的程序以及在 swpdb 控制下运行异构众核程序，具体使用方法见本手册 1 章。

轻量级调试工具集包含 3 个工具：Jdebug 能够对大规模程序的运行状态进行快速查看；Jstack 支持并行程序运行时堆栈的查看；running_state 是基于系统软件接口研制的一款作业级的调试监视工具，已内置在系统环境中，用户无需安装。具体用法见本手册第 2 章。

penv_debug 并行调试工具 GUI 是一款图形化的调试工具，通过图形界面的展示，可以让用户更加直观地看到程序的调试信息，具体用法见本手册第 3 章。

gdb 常用调试命令及部分说明可参考手册附录。

手册中黄色覆盖的红色字体部分如 **//XXXX** 为解释性语言，非输入命令；用户在对程序进行编译时最好添加 -g -O0 选项。

目 录

调试工具用户手册.....	
1 远程调试工具 swpdb	3
1.1 调试正在运行的程序	3
1.2 在 swpdb 控制下运行众核异构程序	5
2 轻量级调试工具集.....	6
2.1 大规模 PC 值扫描工具 Jdebug	6
2.2 并行程序堆栈运行时查看工具 Jstack	6
2.3 轻量级调试监视工具 running_state	8
2.3.1 功能说明	8
2.3.2 使用方法	8
3 并行调试工具图形界面 penv_debug	11
3.1 登录	11
3.2 调试流程	11
4 附录	21
4.1 swpdb 调试命令表	21
4.2 说明	22
5 修改记录	24

1 远程调试工具 swpdb

swpdb 远程调试工具在 gdbserver 的基础上对其进行封装和操作简化，为用户提供了“调试正在运行的程序”和“使用 swpdb 控制下运行程序”两种调试方法，提高了安全性、可用性和调试的效率。

注意：swpdb 目前对于主从混合程序和纯从核程序只能同时显示和调试一个从核线程，在调试时如果需要切换其他从核线程可先使用 `i cg` 命令查看核组及线程信息再使用 `t cg cg_id slave core_id` 命令或 `t s cg_id core_id` 切换到其他从核线程；用户在对程序进行编译时最好添加 `-g -O0` 选项。

用户在使用远程调试工具时可输入 swpdb 回车后查看使用方式，如图 1-1 所示，其参数各选项及含义如下：

jobid: 指定被调试的作业号

-t taskid: 可选项（不指定则使用默认值），指定被调试的任务号，默认值为 0

-n nodeid: 可选项（不指定则使用默认值），指定被调试进程运行节点，默认值为空闲队列中第一个空闲节点号

-r “/bin/swrun ... -I debug_program”: 指定被调试程序的提交命令

```
[swpenv@psn02 ~]$ swpdb
usage:swpdb jobid [-t taskid]
      or
usage:swpdb [-n nodeid] -r "/bin/swrun ..... -i debug_program "
```

图 1-1swpdb 使用方法示例

1.1 调试正在运行的程序

无需登录到节点，直接在本地使用 swpdb JOBID 命令连接到正在运行的程序，使用时最好在程序运行的目录下进行调试。

```
[swpenv@psn01 host_slave]$ swpdb 123293 //连接到作业
.....
(swpdb) i thr
Id Target Id      Frame
2  CG<0>PE<0>    0x00004ffff0410414 in slave__Waiting_For_Task ()
* 1  Thread 1230.1230 "hostslave" main () at form.c:11
(swpdb) bt
#0  main () at form.c:11
Reading in symbols for libc-start.c...done.
(swpdb) l 11
6
7  int main()
8  {
```

```

9      int i=0,j=0,st=1,a[10],b[10];
10     st=1;
11     while(st);
12     //printf("1\n");
13
14     for(i=0;i < 2;i++)
15     {
(swpdb) p st
$1 = 1
(swpdb) p st=0
$2 = 0
(swpdb) n
14     for(i=0;i < 2;i++)
(swpdb) s
16         a[0]=1;
(swpdb) i thr
  Id Target Id      Frame
  2  CG<0>PE<0>      0x00004ffff0410414 in slave__Waiting_For_Task ()
* 1  Thread 1230.1230 "hostslave" main () at form.c:16
(swpdb) t cg 0 slave 2 //切换至上面没有显示出来的从核线程
[Switching to thread 3 (CG<0>PE<2>)]
#0 0x00004ffff0410614 in slave_test () at test.c:11
11     while(st);
(swpdb) i thr
  Id Target Id      Frame
* 3  CG<0>PE<2>      0x00004ffff0410614 in slave_test () at test.c:11
  1  Thread 3540.3540 "hostslave" 0x00004ffff0413d00 in athread_join ()
.....
(swpdb) i cg //查看核组和从核线程信息
CG<0>:
core<0> (stopped) 0x4ffff0410300 in slave_test1() at test.c:31
core<1> (stopped) 0x4ffff0410280 in slave_test1() at test.c:30
core<2> (stopped) 0x4ffff0410390 in slave_test1() at test.c:28
core<3> (stopped) 0x4ffff0410340 in slave_test1() at test.c:32
.....
core<38> (stopped) 0x4ffff0410390 in slave_test1() at test.c:28
core<39> (stopped) 0x4ffff0410340 in slave_test1() at test.c:32
---Type <return> to continue, or q <return> to quit---
Quit
(swpdb) t s 0 38 //切换至对应核组和从核线程
[Switching to thread 3 (CG<0>PE<38>)]
#0 0x00004ffff0410390 in slave_test1 () at test.c:28
28     for(j=0;j<1000;j++)

```

```

(swpdb) bt
#0 0x00004ffff0410390 in slave_test1 () at test.c:28
#1 0x00004ffff04104a4 in slave_test () at test.c:64
#2 0x00004ffff0410664 in slave__Waiting_For_Task ()
Reading in symbols for crtslave.c...done.
(swpdb) i thr
  Id Target Id      Frame
* 3  CG<0>PE<38>    0x00004ffff0410390 in slave_test1 () at test.c:28
  1  Thread 906.906 "hostslave" 0x00004ffff0413b00 in athread_join ()
(swpdb)

```

1.2 在 swpdb 控制下运行众核异构程序

直接使用 `swpdb -n NODEID -r "/bin/swrun ..... -i program"` 命令控制程序运行，其中 `-n` 用于指定程序在某个节点运行，是可选项。

```

$ swpdb -n 53954 -r "/bin/swrun -E 3 -i ./hostslave"
Check the given node .... 53954
Start the remote gdbserver ....
warning: Can not parse XML target description; XML support was disabled at compile time
Reading /bin/sw7run-29 from remote target...
.....
Reading symbols from target:/bin/sw7run-29...done.
Reading in symbols for sw3arun.c...done.
0x00000000120004860 in _start_new ()
(swpdb) bt
#0 0x00000000120004860 in _start_new ()

(swpdb) i thr
  Id Target Id      Frame
* 1  Thread 481.481  0x00000000120004860 in _start_new ()
(swpdb) c
Continuing.
Reading symbols from /home/export/online1/systest/swpenv/luow/gdb_test/host_slave/hostslave...done.
Reading in symbols for form.c...done.
Thread 1 hit Temporary breakpoint -14, 0x00004ffff0413448 in main () at form.c:8
8    {
(swpdb) n
9        int i=0,j=0,st=1,a[10],b[10];

```

注意事项：使用时请确保程序确实可使用 `swrun` 的方式直接提交（比如 MPI 程序是不能直接这么提交的，故不能对 MPI 程序使用该方式进行调试）。

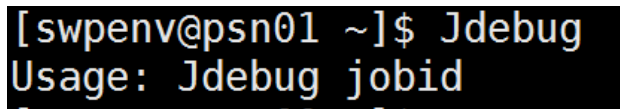
2 轻量级调试工具集

轻量级调试工具集主要实现对正在运行的作业进行信息的扫描和检测，不对程序进行调试也不影响程序的运行，轻量级调试工具主要有三种，分别是Jdebug大规模PC值扫描工具、Jstack并行程序堆栈运行时查看工具、running_state轻量级调试监视工具。大规模PC值扫描工具Jdebug实现对大规模程序的快速状态查看和异常检测，并行程序堆栈运行时查看工具Jstack实现了堆栈信息的查看，轻量级调试监视工具running_state实现了更详细信息的持续监控异常检测。

2.1 大规模PC值扫描工具Jdebug

Jdebug大规模PC值扫描工具通过对正在运行的程序进行三次扫描，并对比各次扫描的结果来对程序的状态进行判断，最后将可能状态异常的函数、PC值以及其他信息打印出来。

Jdebug大规模PC值扫描工具使用方法可直接运行命令后查看，使用方式如下：



```
[swpenv@psn01 ~]$ Jdebug
Usage: Jdebug jobid
```

图 2-1 Jdebug 使用方法示例

使用Jdebug对正在运行的程序进行扫描后，扫描结果会打印到屏幕并输入到本地目录的.Jdebug_jobid.res文件中便于用户查看。在以下例子中可以看到，扫描工具将程序的信息打印输出并在“maybe exception”后面输出可能异常的函数信息。

```
Get and process taskinfo
..... 100.0000%
Fetch and process data
  fetch ..... 100.0000% (3/3)
  classify ..... 100.0000% (15405/15405)
  addr2func ..... finished!
Output to file
..... 100.0000%
===== 格式说明,没有从核号的为主核数据 =====
-> PC值,函数名 计数: (任务号,节点号,核组号[,从核号]) ... (任务号,节点号,核组号[,从核号]) ...
=====
===== maybe exception =====
-> 0x4ff0410200,slave_func 4: (0,18176,0,63) (3,18176,3,63) (41,18265,2,62) (13,18179,1,63)
=====
(gettask 0.102152s) (parsetask 0.000143s) (query 0.837219s) (func 0.000231s) (merge 0.000096s) (outp
k_excp 0.000110s) (output_execep 0.000247s) (all 0.972983s)
```

2.2 并行程序堆栈运行时查看工具Jstack

Jstack并行程序堆栈运行时查看工具，使用方法可输入命令Jstack回车后查看，使用方式如下：

```
[swpenv@psn01 ~]$ Jstack
Usage: Jstack jobid
```

图 2-2 Jstack 使用方法示例

工具使用时在后面直接接上运行的作业号，Jstack jobid 会在命令行界面输出对应的程序堆栈信息，同时弹出图形界面的信息展示窗口。

（注意：在国产一体机上使用本工具之前，使用 ssh 命令连接服务器时需加上 -X 选项才能显示图形界面）

```
[swpenv@psn01 tmp]$ Jstack 1676209
```

```
Collecting data [23]... 100.00%
```

```
Process data ... 100.00%
```

```
=====
```

```
T0 -> 1-89,94-95
```

```
T1 -> 0
```

```
-----
```

```
-0- /home/export/online1/mdt00/systest/swpenv/pdj/test/primary_mpi/./primary_mpi
```

```
-1- main ([0/92])
```

```
-2- PMPI_Barrier ([91/91] T0 94/v81389 95/v81389 48/v81380...)
```

```
-2- pthread_join64 ([1/1] T1 0/v81372)
```

```
=====
```

```
tasks missing stack:(taskid/nodeid)
```

```
90/81389 91/81389 92/81389 ...
```

```
see more in res_file
```

//命令行界面所展示的信息从做往右依次是函数调用栈深度、函数名、调用统计数（方括号中内容，[调用结束在该函数的任务数 / 调用栈经过该函数的任务数]）、方括后跟着任务列表 Txx（Txx：主核线程列表按照作业管理提供的逻辑任务号进行归纳，从核线程列表按照“主核任务号. 从核号”的形成进行归纳展示，上面的例子表示任务 0 和任务 1 的 3 号从核当前在 slave_fun1 函数，其余从核在 slave_fun2 函数）、后续为部分任务的任务号 / 关联的节点号

//部分未获取到函数调用栈的任务（任务号 / 关联的节点号）

//为提高可视化效果，对于大量的任务，界面仅输出有限的任务号及节点，更多数据可在工具使用目录下的.Jstack_[jobid]_[time].res 文件中查看

```
Collecting data [23]... 100.00%
Process data ... 100.00%
```

```
=====
```

```
T0 -> 1-89,94-95
```

```
T1 -> 0
```

```
-----
```

```
-0- /home/export/online1/mdt00/systest/swpenv/pdj/test/primary_mpi/./primary_mpi
```

```
-1- main ([0/92])
```

```
-2- PMPI_Barrier ([91/91] T0 24/v81376 25/v81376 26/v81376...)
```

```
-2- pthread_join64 ([1/1] T1 0/v81372)
```

```
=====
```

```
tasks missing stack:(taskid/nodeid)
```

```
90/81389 91/81389 92/81389 ...
```

```
see more in res_file
```

图 2-3 Jstack 命令行界面信息展示效果

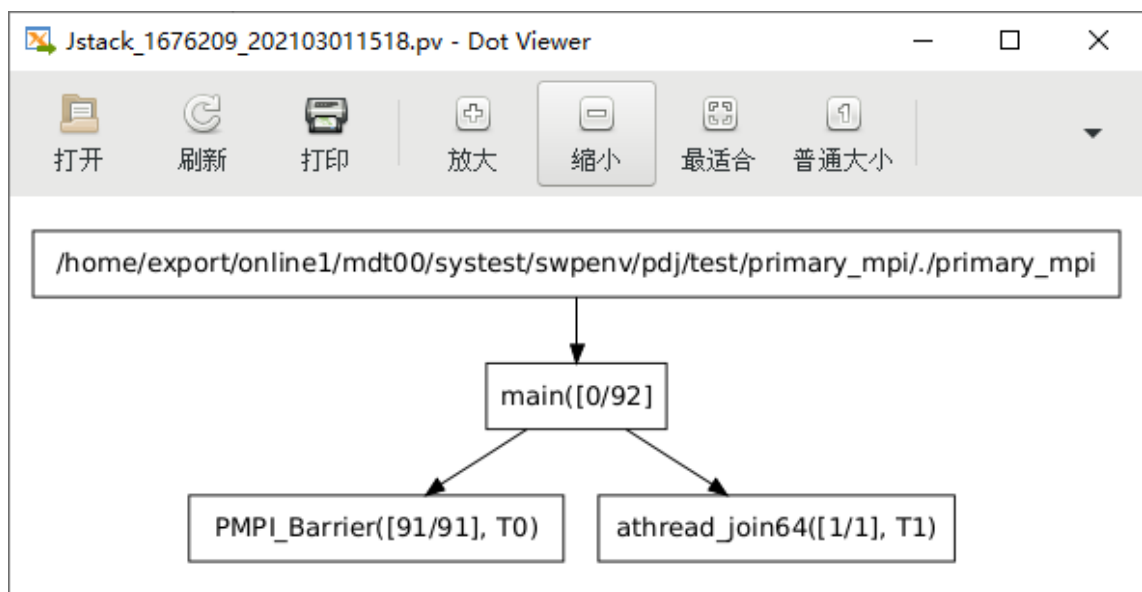


图 2-4 Jstack 图形界面信息展示效果

以图 2-3/2-4 为例进行说明，rankid 为 90-93 的 MPI 任务未获取到函数调用栈，可能这些任务或对应的计算节点存在问题，需要用户处理；分析堆栈调用树，可见 rankid 为 0 的任务在等待从核计算结束，其余任务进入 MPI 同步等待状态，此时需要用户留意 0 号进程可能存在的问题。

2.3 轻量级调试监视工具 running_state

2.3.1 功能说明

running_state 轻量级调试监视工具是一个基于系统软件接口研制的一款作业级的调试监视工具，已内置在系统环境中，无需用户安装，只需要在编译时添加-g 选项。

running_state 能够在程序运行时查看其从核 LDM、gpr、主核程序堆栈、从核当前 pc 和程序行、以及相关硬件性能数据等。

2.3.2 使用方法

running_state 支持单次扫描和持续监测两种使用方式。持续监测模式下，ctrl+c 可退出监测。同时，为了确保堆栈中的信息更具可读性，用户需要添加-g 选项进行编译。

1) 选项说明

输入 running_state 直接回车，会看到工具的使用方法，如图 2-5 所示：

```
[swpenv@psn01 ~]$ running_state
Usage: running_state [-t taskid] [-c coreid] [-m] jobid
```

图 2-5 使用方法示例

其中选项如表 2-1 所示：

表 2-1 工具选项

选项	描述
-t taskid	可选项，选择被监测的任务号，默认监测 0 号任务
-c coreid	可选项，选择被监测的从核号，默认监测 0 号从核
-m	可选项，启动持续监测
jobid	必选项，被监测程序的作业号

2) 结果说明

结果如图 2-6 所示，大致分为四个部分：从核局存（LDM）、从核通用寄存器（GPR）、从核性能数据、堆栈（Stack）。窗口中仅输出了部分内容，前两部分仅显示了指定从核的信息，后两部分显示了指定从核同一行的 8 个核的信息，更详细信息请查看结果文件 resdat.txt。

LDM 部分，从左往右依次是偏移地址、数据和数据对应的 ASCII 码字符。

GPR 部分，从左到右、从上到下依次是 0~31 号通用寄存器。

从核性能数据包含了 11 列，如下表所述，除 id 和 gpc 外的数据仅在监控模式下有意义。

堆栈部分包含了主核的堆栈和每个从核当前对应的文件名及行号。

表 2-2 从核性能数据描述

列名称	描述
id	从核号
gpc	从核 PC 值
comm (MB)	节点通信数据
io (MB)	节点 IO 数据
cycle	从核周期计数
inst	从核执行指令计数
llic_acc	从核 L1Cache 访问计数
llic_mis	从核 L1CacheMiss 计数
gld	从核 gld 计数
vec_int	从核执行向量整数运算指令数
vec_f	从核执行向量浮点运算指令数（含 VLENMAS）

单次扫描模式下仅会输出一组结果，持续监测模式下会 1 秒左右刷新一次，直至被 ctrl+c 终止。

```

----- taskId:0 coreid:0[ the 10th scan ] (Details in the file resdat.txt) -----
==LDM==
00000000 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
+
00000100
==GPR==
(t0~t7 )          16          15520044949842981407073924575765308018524160          12          25          1
(t8~t12) 70368744331264 87960666047852          155200          155200          155200
(s0~s6 )          1          64          552          520          0          0          155072
(a0~a5 ) 87960930400884 87960930408064          157992          163680          0 87960930400884
(v0)155496, (fp)155072, (ra)4ffff0410664, (pv)155200, (AT)1, (gp)500000031070, (sp)155072
==slave(id, gpc, comm(MB), io(MB), cycle, inst, llic_acc, llic_mis, rmemacc, vec_int, vec_f)==
0,4ffff0410c20,12940.69, 0.00, 19643231094, 10624254803, 2812152766, 87703, 11035291, 63784, 0
1,4ffff0410b40,12940.69, 0.00, 19643227595, 10624251387, 2812153035, 87697, 11035305, 63780, 0
2,4ffff0411130,12940.69, 0.00, 19643224484, 10621379748, 2811396494, 87720, 11032972, 63796, 0
3,4ffff0410a00,12940.69, 0.00, 19643221368, 10621544229, 2811437966, 87723, 11032908, 63800, 0
4,4ffff0410c10,12940.69, 0.00, 19643218249, 10621093416, 2811315451, 87663, 11031738, 63754, 0
5,4ffff0410a00,12940.69, 0.00, 19643089412, 10620694652, 2811212844, 87704, 11031940, 63784, 0
6,4ffff0410a40,12940.69, 0.00, 19643344805, 10619397939, 2810870398, 87682, 11030620, 63768, 0
7,4ffff0411470,12940.69, 0.00, 19643341689, 10619438252, 2810879480, 87657, 11030333, 63752, 0
==Stack==
#0 (0x4ffff0410c20 implschs_part1_ slave-thafts.f90:118)
#1 (0x4ffff041036c implschs_part1_ slave-thafts.f90:1)
[0] (slave-thafts.f90:118) [1] (slave-thafts.f90:118) [2] (slave-thafts.f90:130) [3] (slave-thafts.f90:114)
[4] (slave-thafts.f90:118) [5] (slave-thafts.f90:114) [6] (slave-thafts.f90:114) [7] (slave-thafts.f90:138)

```

图 2-6 结果示例

3 并行调试工具图形界面 penv_debug

3.1 登录

- 1、通过 XShell 登录 SN 节点,或者通过实体机的终端执行 `ssh -X username@ip` 登录 SN 节点
- 2、执行 `penv_debug` 命令,弹出并行调试工具客户端,如图 3-1 所示:

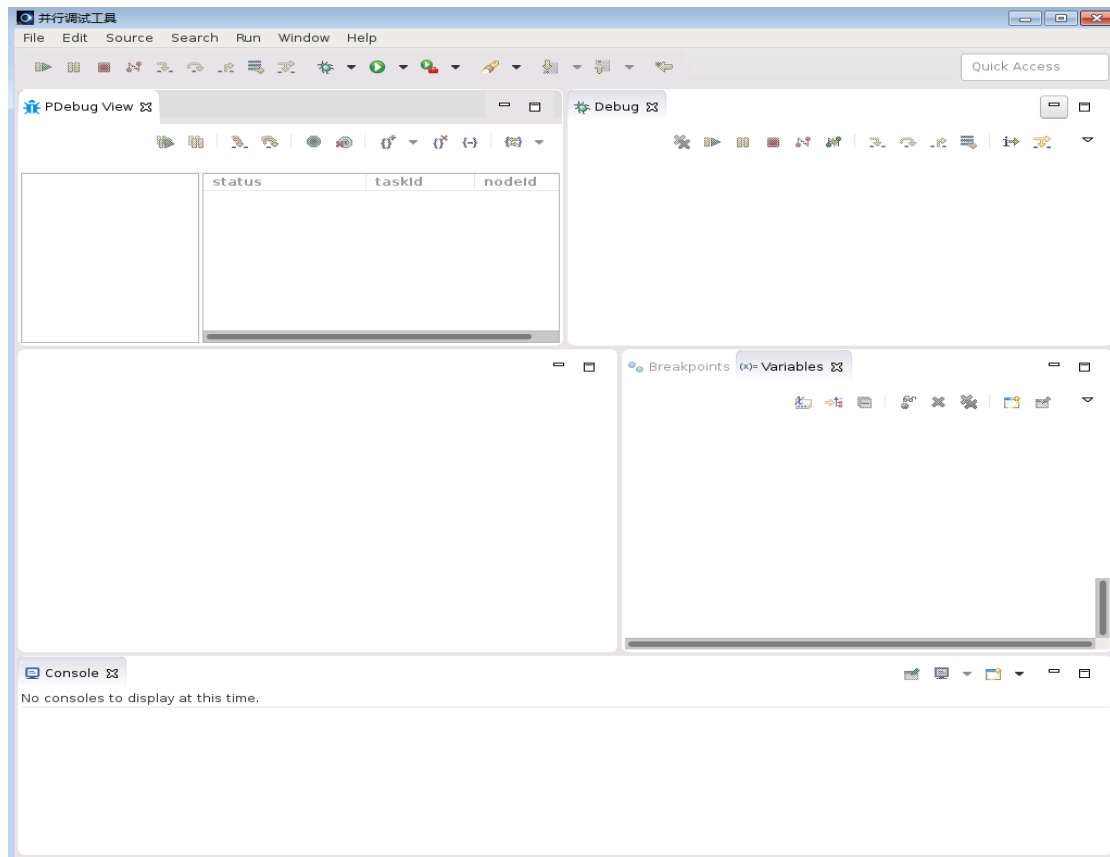


图 3-1 并行调试工具客户端界面图

3.2 调试流程

1、启动调试

(注意被调试的程序编译时需使用 `-g -O0` 参数。)

- A、自动模式: 鼠标点击工具栏菜单 "File" -> "Quick Debug Job", 如下图 3-2 所示:

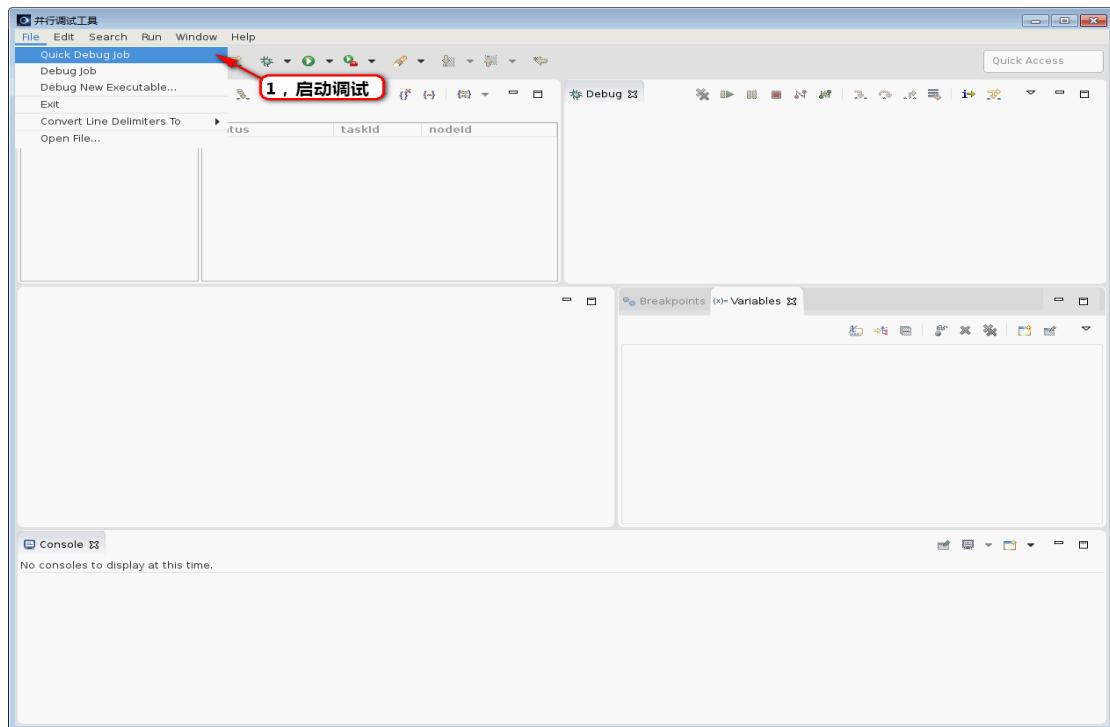


图 3-2 启动调试图

B、手动添加路径，鼠标点击工具栏菜单"File"->" Debugjob", 选择二进制文件放置的位置，如图 3-4 和图 3-4 所示：

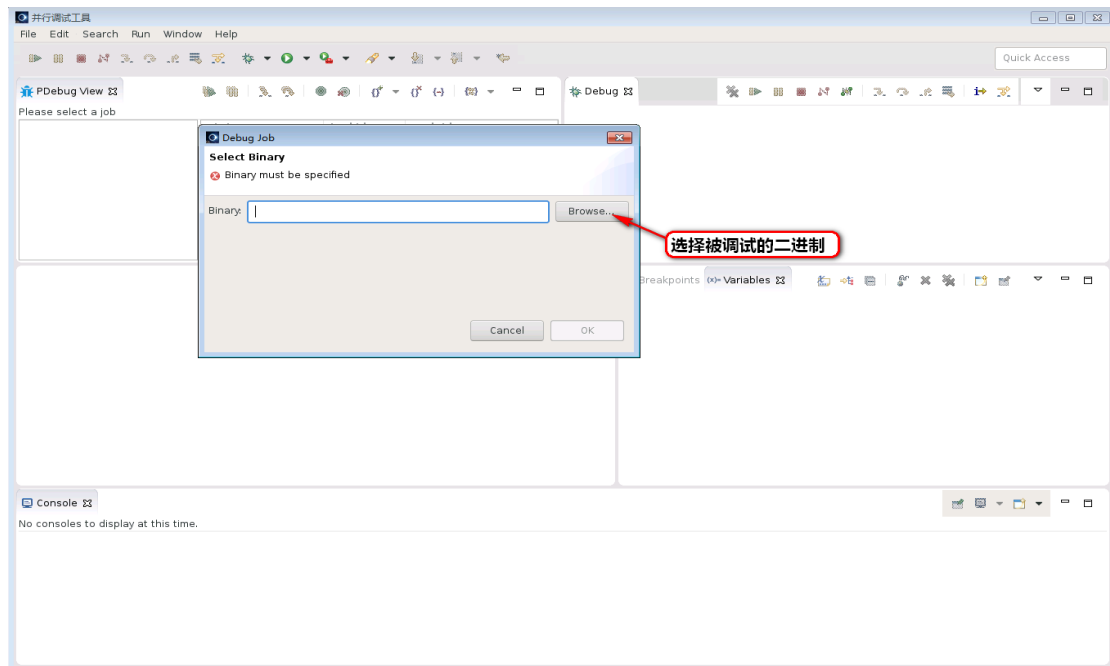


图 3-3 手动添加

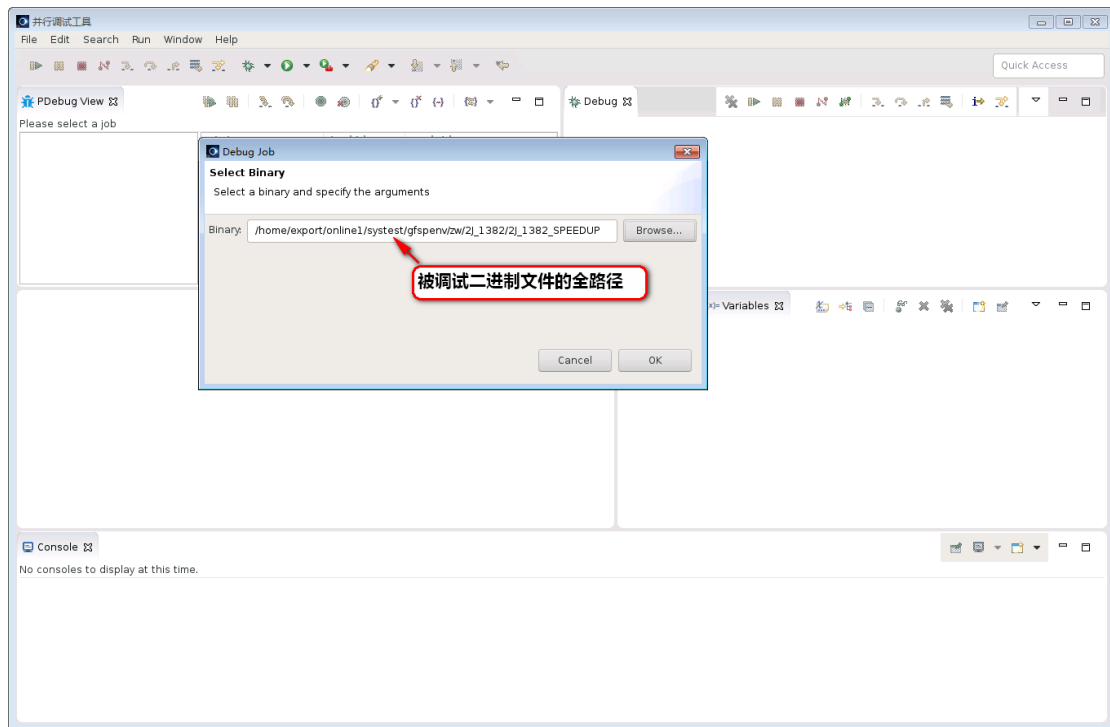


图 3-4 手动添加

C、启动调试成功，弹出作业列表的对话框，选中相应的作业号，点击"OK"按钮，启动调试。如图 3-5 所示：

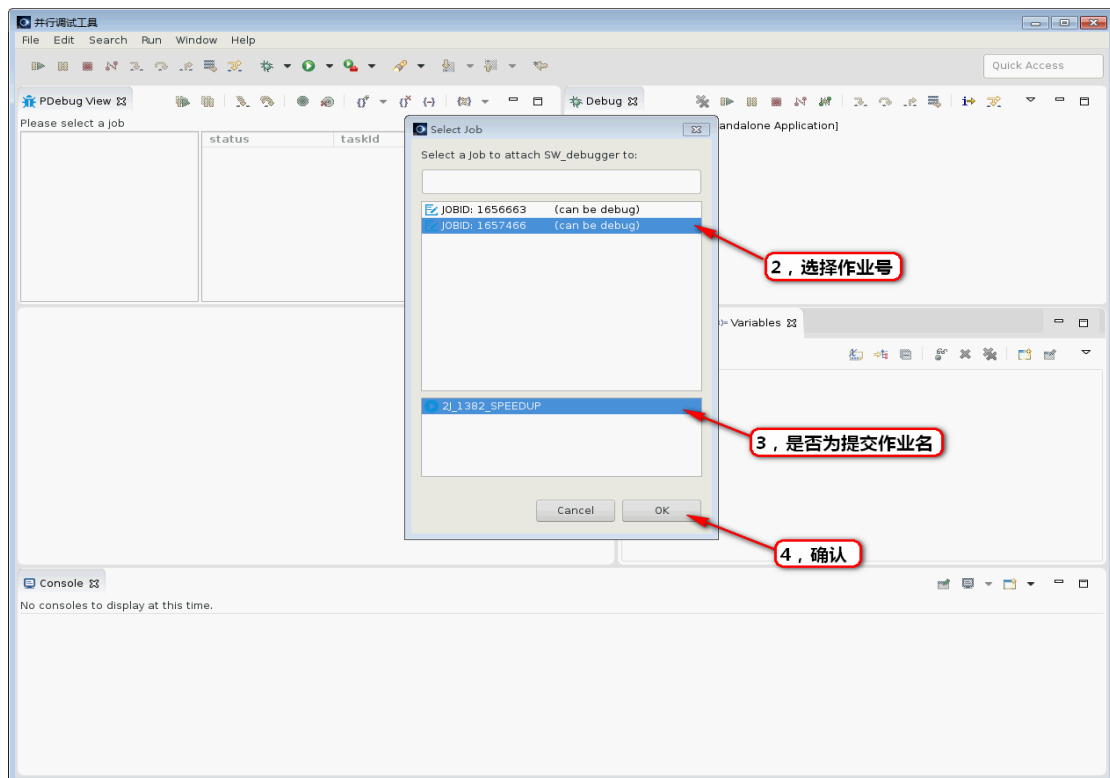


图 3-5 作业列表图

2、调试透视图

透视图包括了并行调试视图、调试视图、源码显示区域、断点视图、变量视图、控制台

视图如图 3-6 所示：

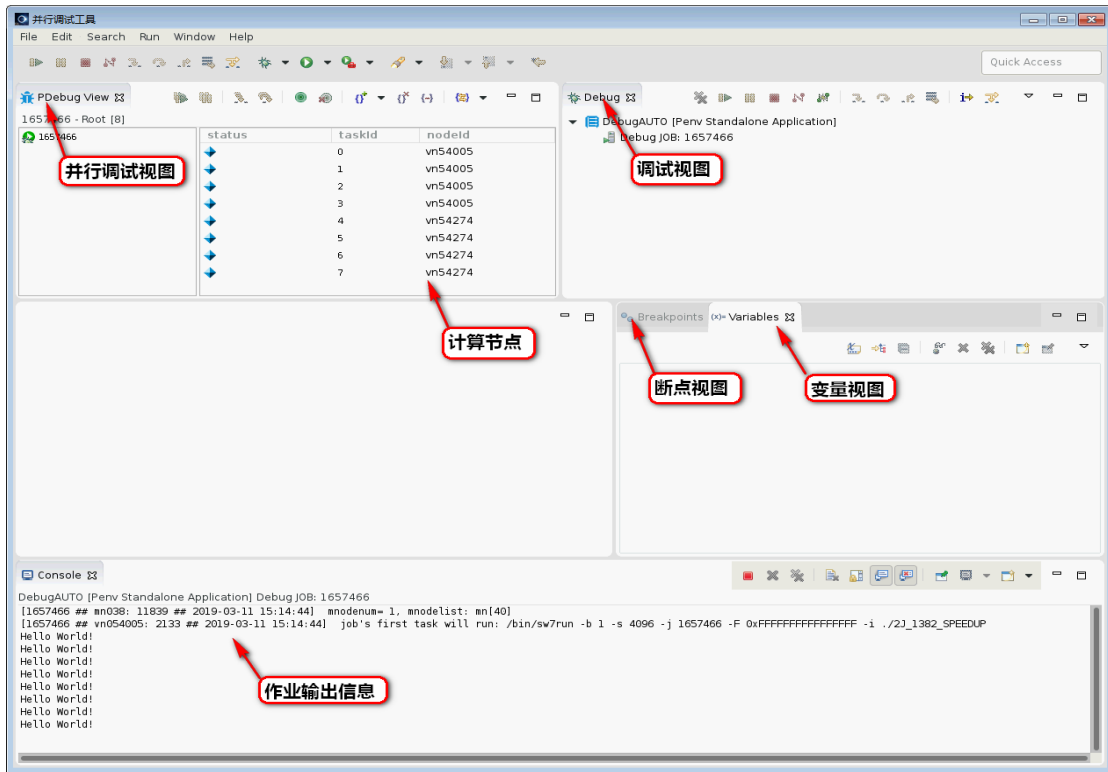


图 3-6 并行调试透视图

3、调试命令相关

鼠标点击"Attach"图标，把 GDB 附加到进程上，如图 3-7 所示：

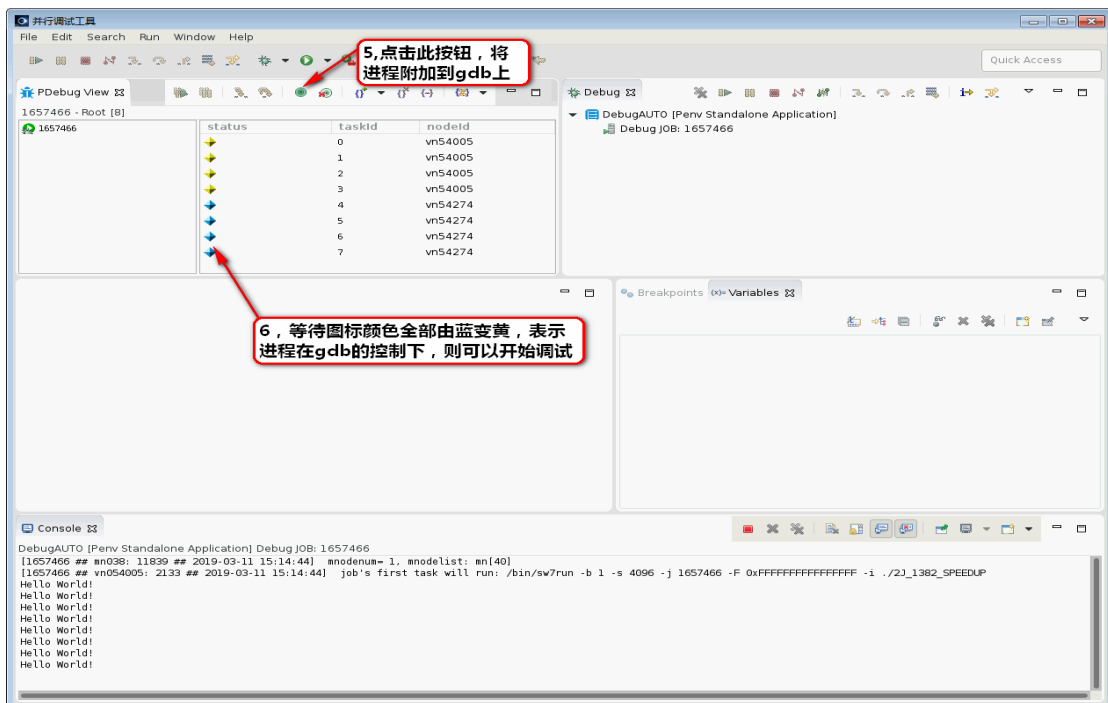


图 3-7 附加 GDB 到相应进程上

执行"Attach"后，进程图标颜色由蓝变黄，此时说明 GDB 已经附加到进程，并且进程当前状态为中断状态；这时相应的并行调试命令 RESUME、STEP INTO、STEP OVER 为可用状态，待全部进程变为黄色时可以开始使用。如图 3-8 所示：

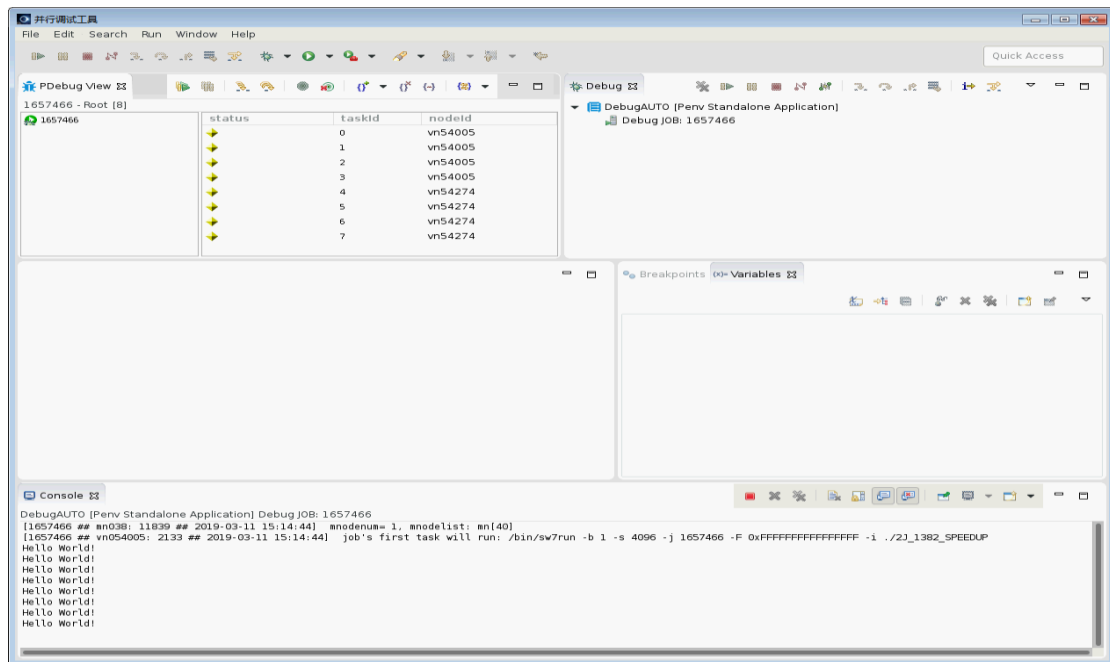


图 3-8 加载完成

鼠标选中某一进程双击，即可把该进程注册到调试视图，调试视图可看到该进程的栈信息；相应的变量视图对应着具体栈中的局部变量，源码显示区域定位到行；如图 3-9 所示：

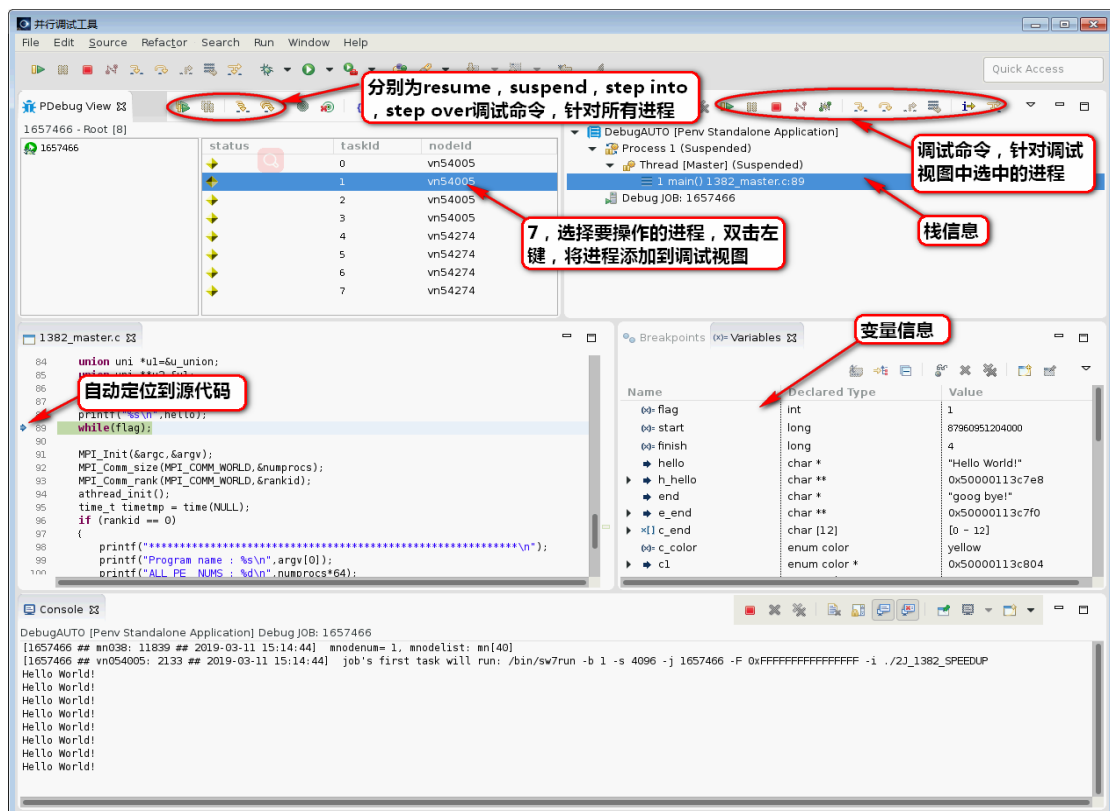


图 3-9 调试进程

鼠标定位到编辑器的左边侧栏右键->"Breakpoint Type->Penv Breakpoints"选择断点类型，如图 3-10 所示：

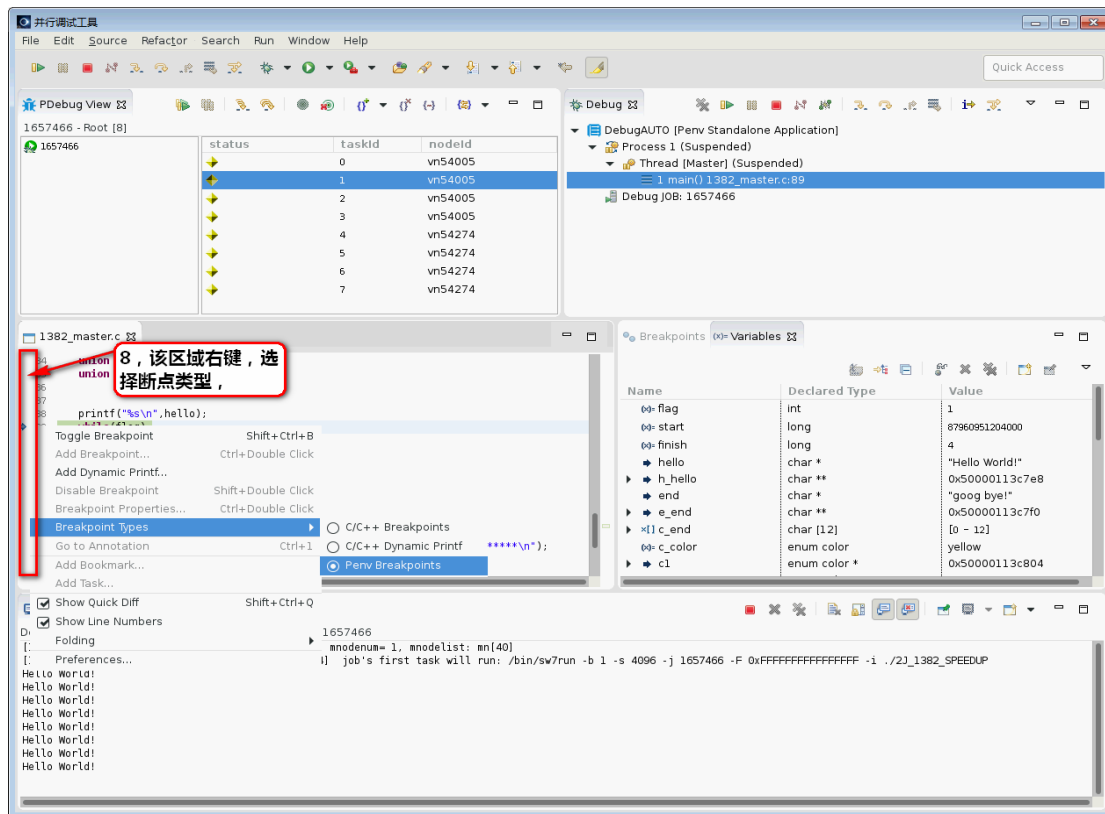


图 3-10 断点类型设置

鼠标定位到编辑器的左边侧栏右键->"Toggle Breakpoint";设置断点;如图 3-11 所示:

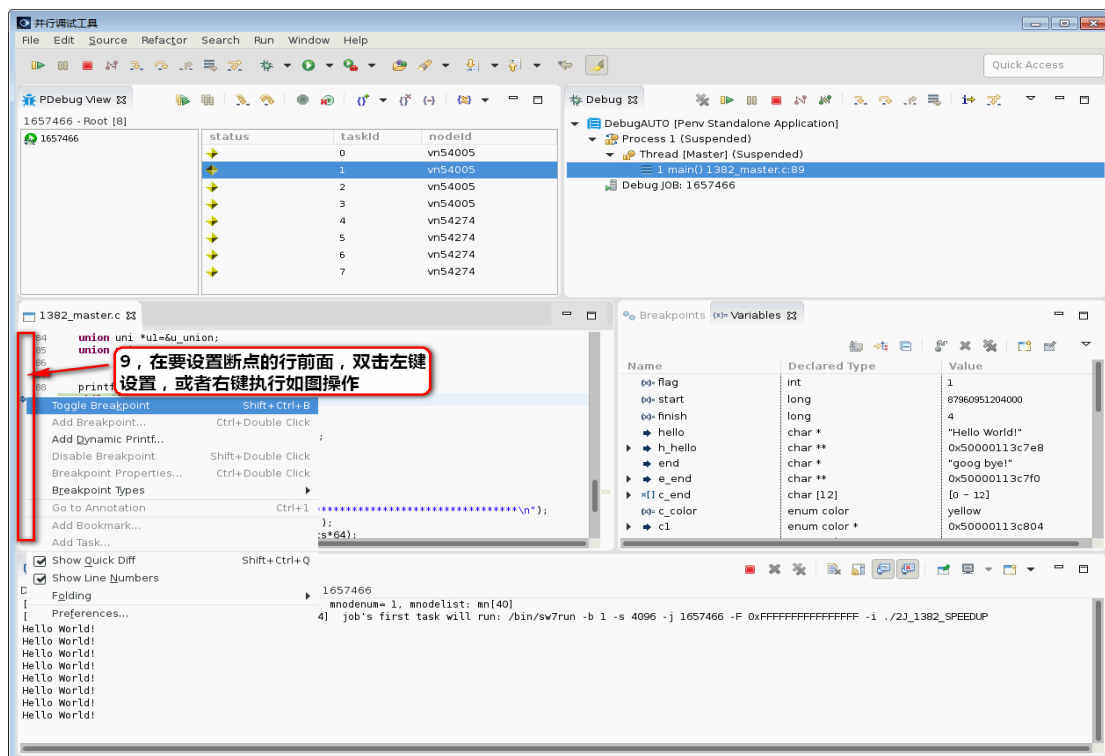


图 3-11 设置断点

设置断点后, 编辑器上显示断点信息并且断点视图中也出现相应断点信息; 如图 3-12 所示:

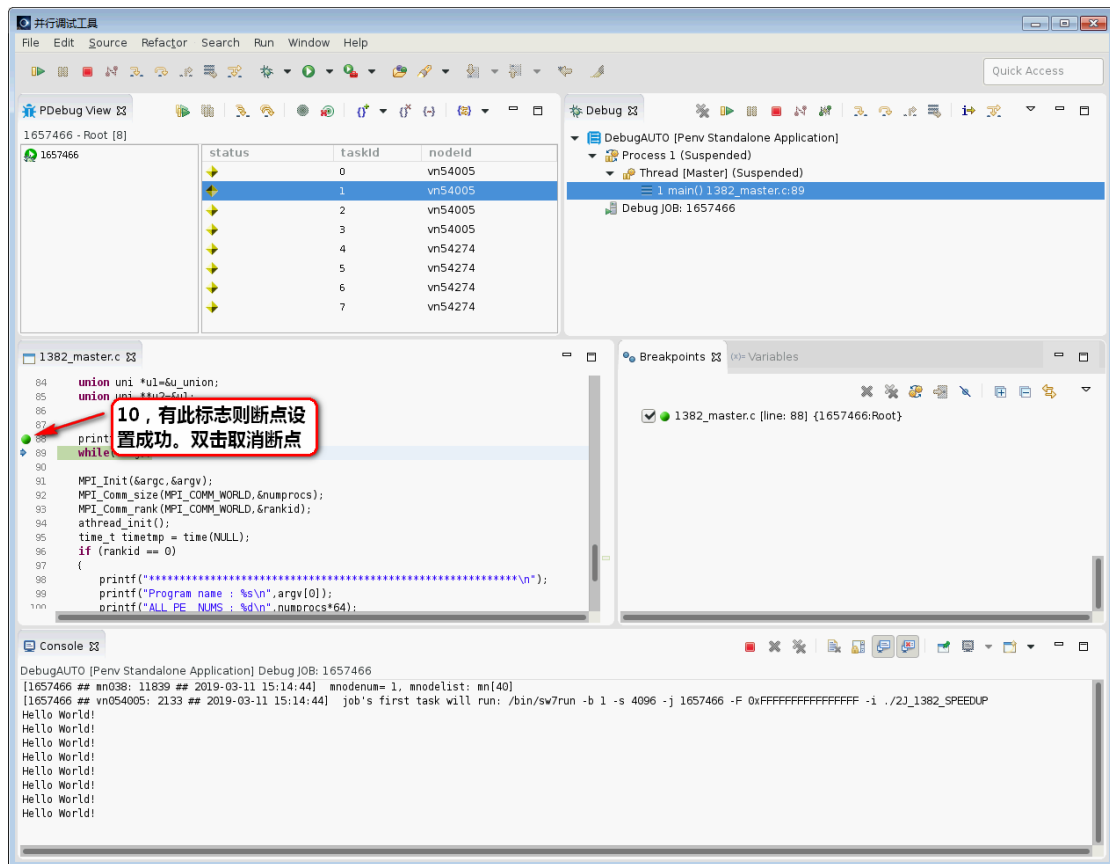


图 3-12 断点设置成功视图

查看程序中的变量名，变量类型，变量值，如图 3-13 所示：

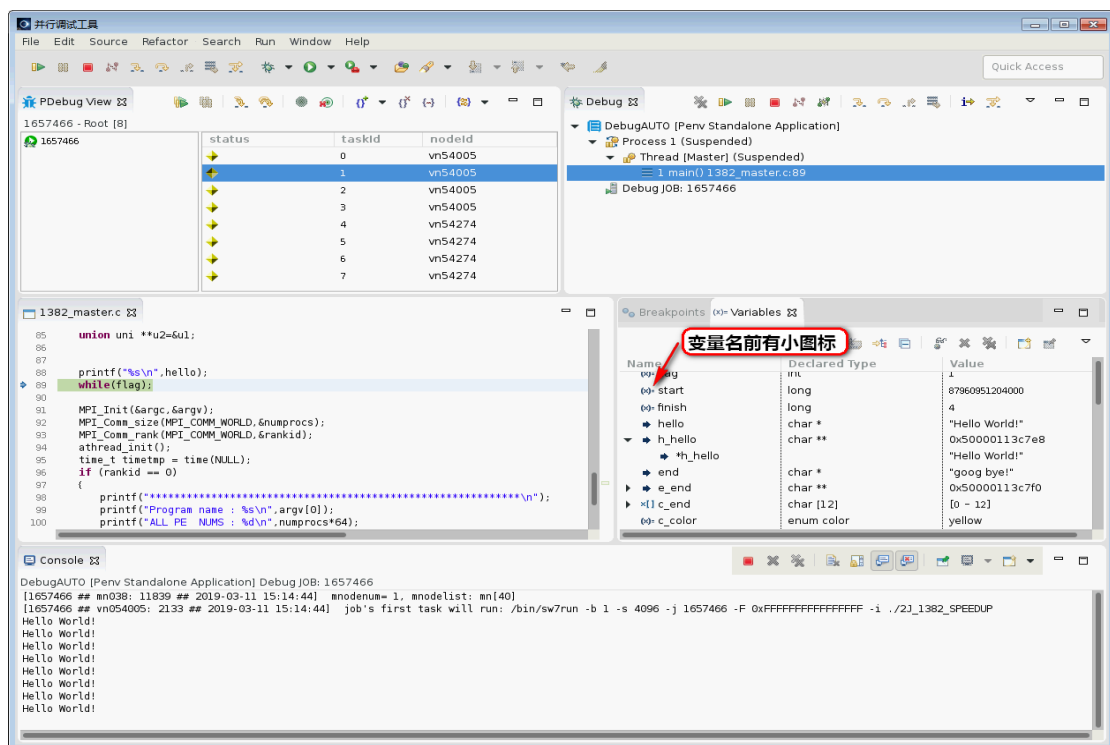


图 3-13 查看变量

上图中，变量名前面有 4 种不同的小图标， 表示结构体、类， 表示数组， 表示指针， 表示普通变量。

图标前的小三角表示可以查看更多信息，点击小三角图标，结果如图 3-14 所示：

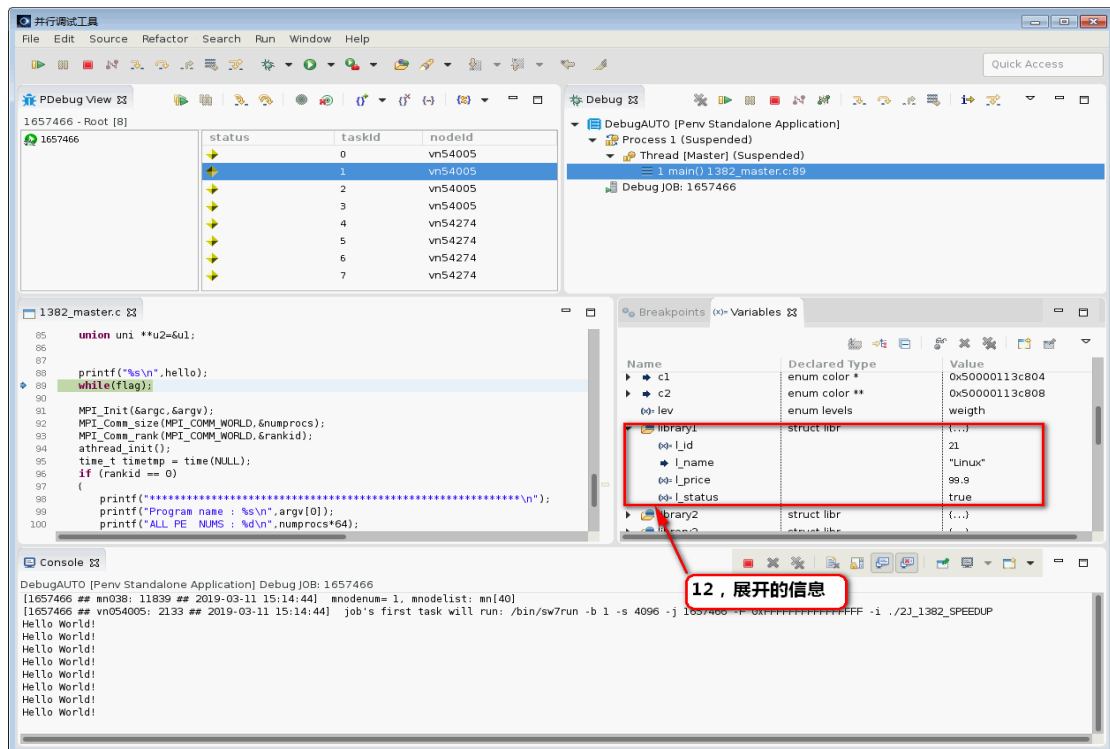
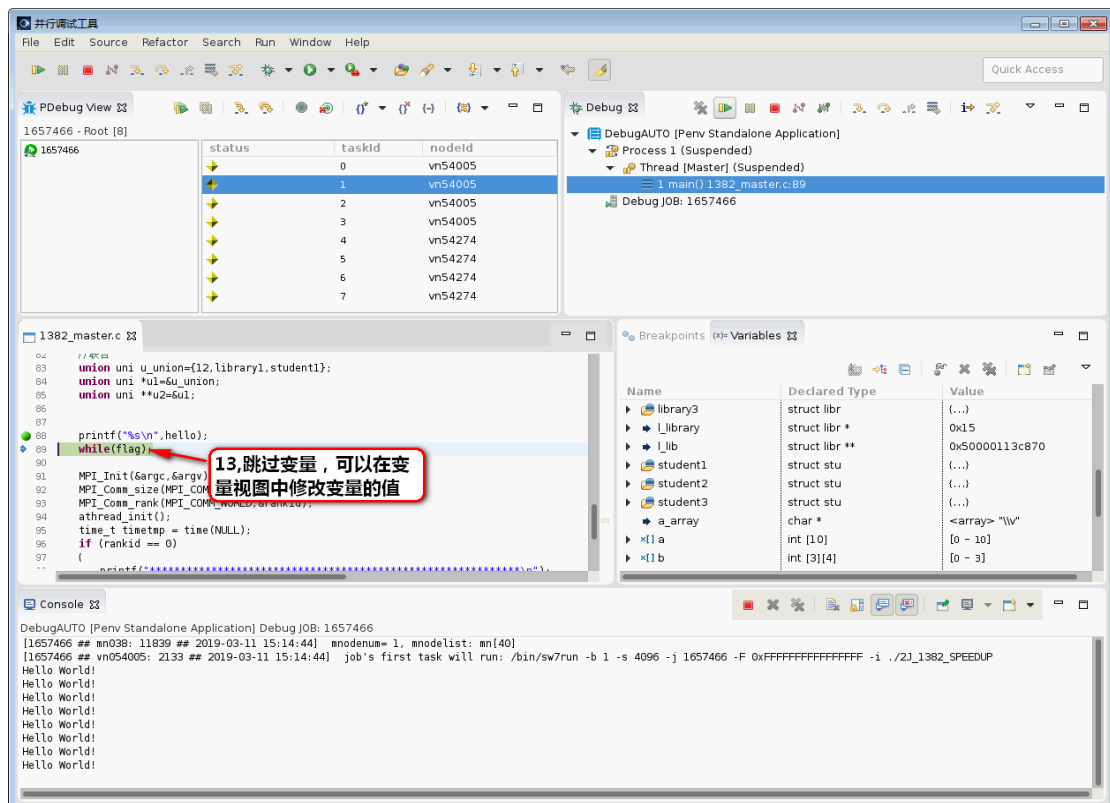


图 3-14 变量展开图

可以通过在变量视图中修改变量的值，如图 3-15 所示：



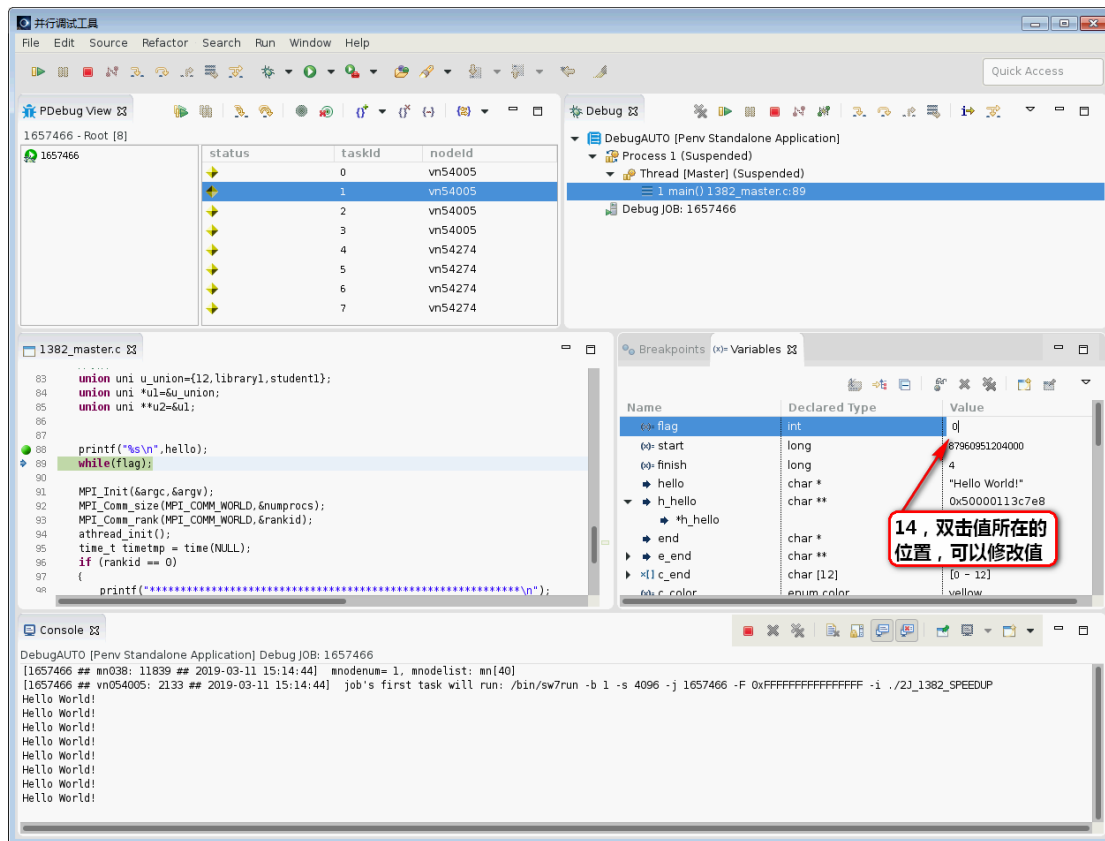


图 3-15 修改变量值

双击运行第二个进程，修改第二个进程 flag 变量后，图标颜色由黄变绿，此时进程为 running 状态，如图 3-16 所示：

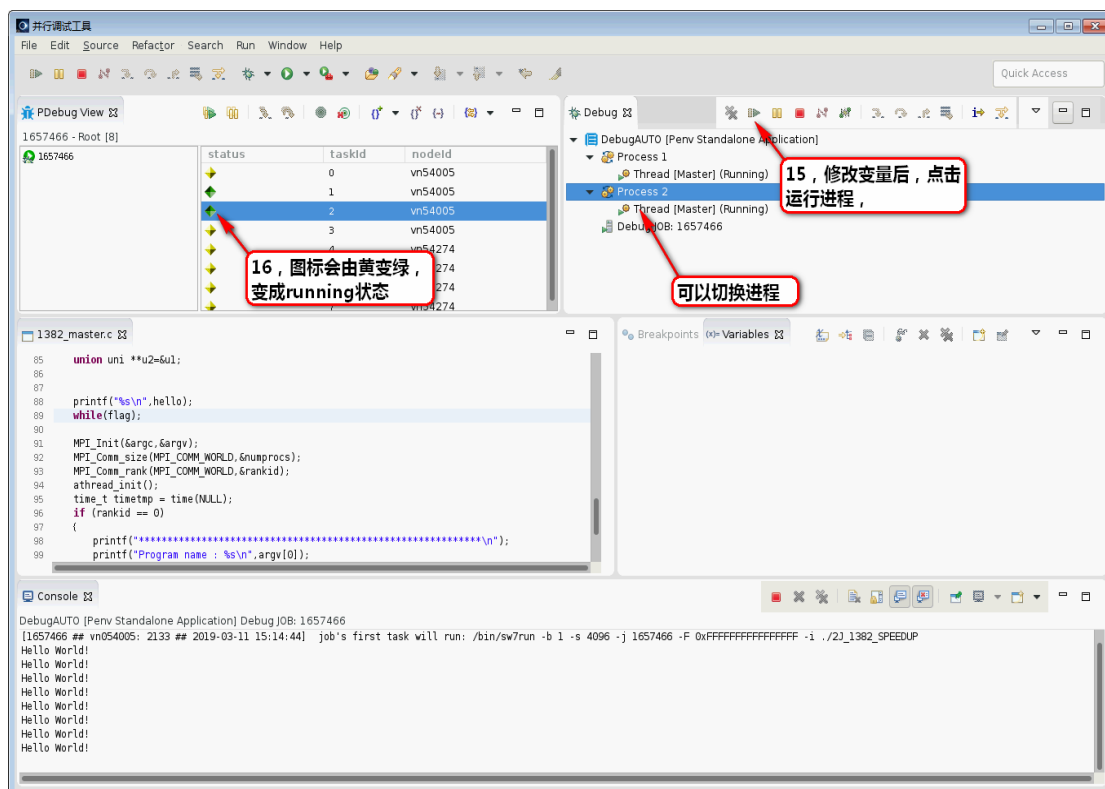


图 3-16 进程状态切换

进程状态由绿变黄，从 running 状态变为 interrupt 状态，如图 3-17 所示：

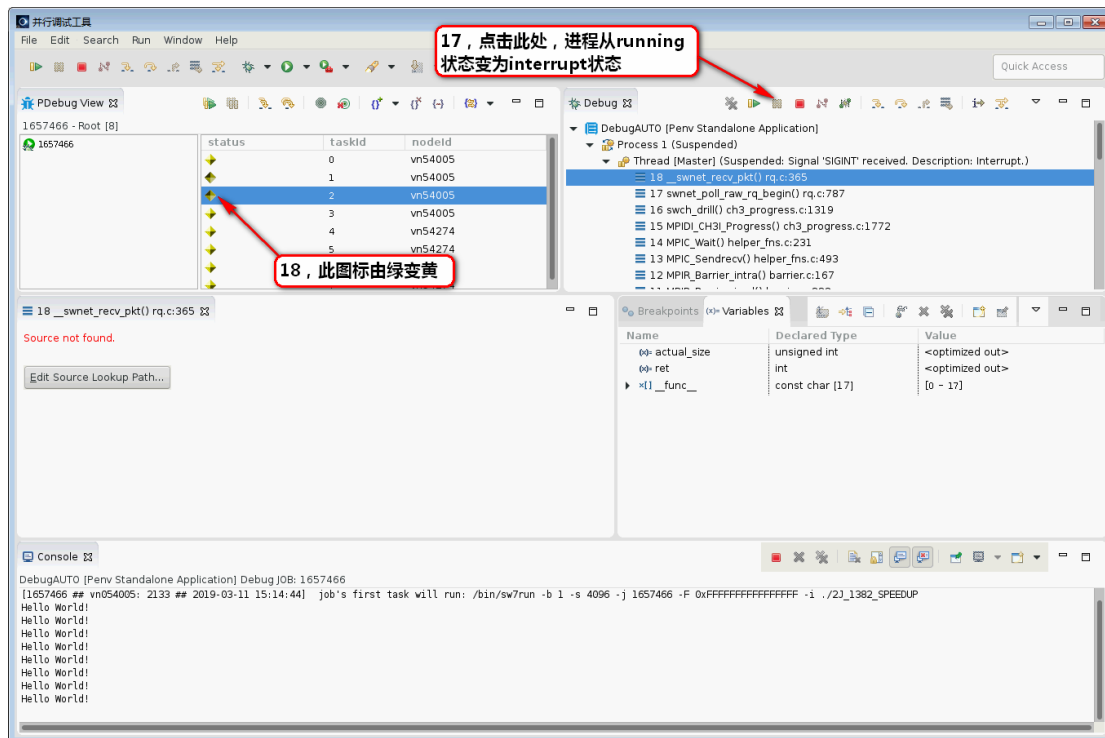


图 3-17 退出 running 态
调试结束, 退出调试状态, 如图 3-18 所示:

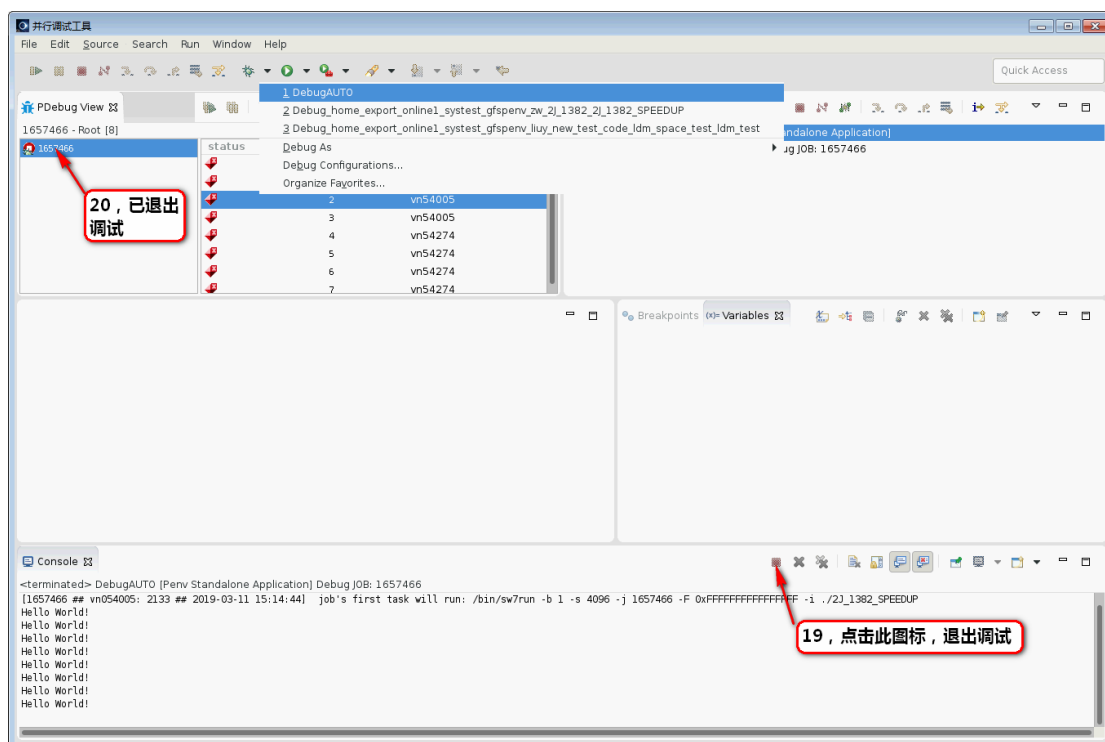


图 3-18 退出调试与重启

4 附录

4.1 swpdb 调试命令表

命令简写	命令语法	功能
sym	symfile exec-file	为调试器提供变量、行号的调试信息
i thr	info threads	显示被调试的线程表
t tid	thread tid	指定线程表中第 tid 号线程为当前调试线程
i reg	info regs sym	显示当前调试线程的所有通用寄存器
bt	where	显示当前调试线程的函数调用栈树
p var	print var	打印变量，输入 set print pretty 命令可优化数组、结构等变量的打印格式
x/fmt address	x/FMT ADDRESS	检查内存块，ADDRESS 虚地址 FMT 是一个十进制数后跟两个字符：格式字符和大小字符。 格式： o(octal), x(hex), d(decimal), u(unsigned decimal), t(binary), f(float), a(address), i(instruction), c(char) and s(string). 大小： b(byte), h(halfword), w(word), g(giant, 8 bytes).
s	step num	执行 num 次单步，num 可省略，缺省为 1 次
n	next num	同 step
c	cont signal	执行，并给程序发送信号，signal 一般省略，即不发送信号

主核	命令简写	命令语法	功能
	wa	watch 变量 watch *地址	写访问主核变量或地址时，停止程序执行
	rw	rwatch 变量 rwatch *地址	读访问主核变量或地址时，停止程序执行

	aw	awatch 变量 awatch *地址	读、写访问主核变量或地址时，停止程序执行
	vw	vwatch 变量	发生变量值被写入时，停止程序执行，可与 wa 一起使用，详见 1.2 节
从核	wa/rw/aw	同主核	访问从核变量或地址时，通知用户，程序不停。
注：	主核只能设置一个变量或地址的匹配，从核可设置两个变量或地址的匹配 从核不支持 vwatch 观察点(值匹配)		

	命令简写	命令语法	功能
主 从核	b	break 文件名: 行号 break 函数名 break * 地址	设置断点
	tb	tbreak ...(同上)	设置一次性临时断点，断点到达时自动删除
	i b	info break	列出断点表
	disable	disable 断点号	使断点无效，但保留在表中
	d	del 断点号	删除断点
	enable	enable 断点号	使断点有效，
从核	hb	hbreak ...(同 break)	设置从核硬件断点
注：	断点表中的从核硬件断点的数量不超过两个。		

4.2 说明

用户一般对正在运行的程序进行 attach，以获得主从核的函数调用栈信息，打印程序变量从而判断程序运行状态。在此建议用户：

1、程序要使用 -g -O0 的编译选项，否则 gdb 只能定位到函数，不能定位到行号，也无法打印局部变量。

2、软件强制硬件从核组暂停的状态是不稳定的，已经发射的指令并未完全暂停执行，硬件寄存器处于非稳态，这时看到从核即将取指执行的位置，不是暂停的位置。如果用户希望在 `gdb` 中继续跟踪执行从核，先设置一个从核断点，等从核到达断点之后（进入稳态），再进行通常的断点、单步等调试操作。

3、由于从核硬件资源的限制，只有 2 个从核硬件断点有效，多出的断点将被覆盖。主核程序不存在稳态和资源限制，被软件暂停的主核可以执行单步，对主核的调试没有过多的限制。

4、用户可忽略 `swpdb` 的所有警告信息，如 `warning: ...`。若 `gdb` 输出“`Cannot access memory at address ...`”，一般表示 `swpdb` 无法定位到一个没有访问权限的地址，程序的栈空间或被程序自己写坏。

5 修改记录

版本	说明	撰写人/审校人	时间
V1.0	建立初始版本。	冯赞龙/刘勇	2020-05-14
V1.1	完善本地调试器、running_state、图形界面 penv_debug 等工具；根据测试情况修改手册中不一致的地方。	张威、冯赞龙/刘勇	2020-05-29
V1.2	2.2 节增加注意事项； 去掉作业管理-debug 选项的相关描述。	冯赞龙/刘勇	2020-07-08
V1.3	删除原有第 1 章节本地调试器内容，其余章节顺延；图片进行相应调整	冯赞龙/刘勇	2020-12-11
V1.4	增加第 1 章中 swpdb 各参数的描述	冯赞龙/刘勇	2021-01-26
V1.5	修改 swpdb 使用方式（无需-j 选项）及相关截图； 修改 Jstack 分析结果展示描述	冯赞龙、彭达佳/刘勇	2021-03-01